COMPTE RENDU DU TP 1

1. Objectif et environnement de travail

Ce travail pratique consiste à réaliser un mini-coffre-fort cryptographique en Python. L'objectif est de protéger des données à l'aide d'un mot de passe, tout en assurant leur confidentialité et leur intégrité. Le programme assemble des primitives reconnues plutôt que de concevoir un nouvel algorithme : Argon2id pour dériver une clé depuis le mot de passe, HKDF-SHA256 pour produire une sous-clé et AES-256-GCM pour chiffrer et authentifier les données. Le projet a été développé sous Ubuntu avec Python 3 et les bibliothèques argon2-cffi, cryptography et pytest.

2. Choix des paramètres et dérivation des clés

Les paramètres retenus pour Argon2id sont T = 30 passes, M = 65 536 Kio (64 Mio) et P = 4 fils. Les mesures réalisées sur la machine de développement donnent un temps médian d'environ 0,494 seconde pour T = 30, ce qui est proche de l'objectif indicatif de 0,5 seconde fixé dans le sujet. Cette durée dépend toutefois du matériel utilisé.

La fonction valider_parametres_argon2() vérifie les bornes autorisées pour le temps, la mémoire et le parallélisme, ainsi que la cohérence entre mémoire et parallélisme. Cette vérification est importante puisque les paramètres lus dans un fichier ne doivent pas pouvoir imposer une dérivation démesurément coûteuse. La fonction cle_maitresse() encode le mot de passe en UTF-8 et utilise Argon2id avec un sel aléatoire de 16 octets pour produire une clé de 32 octets. Ensuite, sous_cle() applique HKDF-SHA256 avec l'étiquette « coffre-v1|chiffrement » afin de dériver une clé dédiée au chiffrement.

3. Chiffrement, déchiffrement et format CFR1

À chaque chiffrement, la fonction chiffrer() génère un nouveau sel de 16 octets et un nonce de 12 octets avec os.urandom(). Elle construit ensuite un en-tête de 44 octets : signature CFR1 (4 octets), paramètres Argon2id (12 octets), sel (16 octets) et nonce (12 octets). Les paramètres sont encodés en entiers de 32 bits en ordre big-endian. Après dérivation des clés, AESGCM chiffre les données et ajoute un tag d'authentification de 16 octets.

L'en-tête complet et le nom du fichier encodé en UTF-8 constituent les données associées authentifiées (AAD). Ils restent visibles, mais toute modification est détectée au déchiffrement. La fonction dechiffrer() contrôle la taille minimale du blob, la signature CFR1 et les paramètres Argon2id, puis reconstitue les clés et l'AAD. Si le mot de passe est incorrect ou si le contenu, l'en-tête ou le nom associé a été altéré, AES-GCM rejette l'opération en levant notamment InvalidTag.

4. Tests automatisés et résultats attendus

Test réalisé	Résultat attendu	Propriété vérifiée
Aller-retour	Le message déchiffré est identique à l'original.	Correction du chiffrement/déchiffrement
Deux chiffrements	Deux blobs du même message sont différents.	Sel et nonce aléatoires
Mot de passe erroné	Une exception InvalidTag est levée.	Authentification
Donnée chiffrée modifiée	Une exception InvalidTag est levée.	Intégrité
Nom de fichier modifié	Une exception InvalidTag est levée.	Liaison au contexte par l'AAD
Rechiffrement	Le contenu est conservé et le nouveau blob diffère.	Changement de mot de passe
Paramètres excessifs	Une exception ValueError est levée.	Validation des paramètres

Les sept tests présents dans test_coffre.py couvrent les cinq vérifications minimales demandées dans le sujet, ainsi que le changement de mot de passe et le rejet de paramètres Argon2id excessifs. La commande d'exécution est : python -m pytest -v. Le fichier README décrit les tests et leur comportement attendu.

5. Évolution : changement du mot de passe

La fonction rechiffrer() commence par déchiffrer le blob avec l'ancien mot de passe. Si cette opération réussit, elle chiffre de nouveau les données avec le nouveau mot de passe. Le nouveau fichier possède donc un nouvel en-tête, un nouveau sel et un nouveau nonce. Le format CFR1 conserve les paramètres Argon2id dans l'en-tête : le programme peut ainsi retrouver les paramètres utilisés pour chaque fichier et faire évoluer les réglages sans rendre automatiquement les anciens fichiers illisibles, à condition de continuer à accepter leurs paramètres.

6. Réponses aux questions de réflexion

Pourquoi le sel est-il stocké en clair ? Le sel n'est pas une clé et n'a pas besoin d'être secret. Il permet de produire une dérivation propre à chaque fichier, même lorsque deux fichiers utilisent le même mot de passe, et limite l'intérêt des tables précalculées. Sa publication ne compromet donc pas, à elle seule, la sécurité.

Pourquoi le nonce aléatoire ne pose-t-il pas ici de problème de répétition ? Le nonce doit être unique pour une même clé AES-GCM. Dans ce programme, chaque fichier reçoit un sel aléatoire, ce qui conduit normalement à une clé dérivée différente. Le risque de réutiliser le même couple clé/nonce est ainsi fortement réduit. Il ne faut toutefois jamais réutiliser volontairement un nonce avec une même clé.

Que se passerait-il si le nom du fichier n'était pas dans l'AAD ? Un attaquant pourrait déplacer ou renommer un fichier chiffré sans que le mécanisme d'authentification détecte ce changement de contexte. L'inclusion du nom dans l'AAD lie donc les données au nom attendu.

Pourquoi authentifier l'en-tête ? L'en-tête contient les paramètres de dérivation, le sel et le nonce. Son authentification empêche un attaquant de modifier ces éléments sans déclencher un échec de vérification. La validation préalable des paramètres complète cette protection en limitant les valeurs acceptées.

Pourquoi ne pas afficher la cause précise d'un échec ? Un message différent selon que le mot de passe est faux ou que le fichier a été altéré pourrait fournir des indices à un attaquant. L'application devrait donc afficher un message générique, par exemple : « Impossible de déchiffrer les données : mot de passe incorrect ou fichier altéré. »

7. Limites et conclusion

Le mini-coffre-fort manipule des données sous forme d'octets en mémoire : il ne fournit pas encore d'interface pour sélectionner, lire et enregistrer automatiquement des fichiers sur le disque. Il s'agit donc d'un prototype pédagogique. Le mot de passe doit rester secret, les clés dérivées ne doivent pas être exposées et les paramètres Argon2id doivent être contrôlés avant leur utilisation.

Ce TP a permis de mettre en œuvre une chaîne cohérente de protection : Argon2id renforce la dérivation du mot de passe, HKDF sépare les usages de clés et AES-256-GCM assure simultanément le chiffrement et l'authentification. Le format CFR1 stocke les informations nécessaires au déchiffrement tout en protégeant l'en-tête et le nom du fichier par l'AAD. Enfin, les tests pytest vérifient les principaux comportements attendus et démontrent les propriétés fonctionnelles et de sécurité du mini-coffre-fort.